UNIDAD 5 – AJAX

DESARROLLO WEB EN ENTORNO CLIENTE

Índice

- ¿Qué es AJAX?
- AJAX XMLHttp.
- AJAX request.
- AJAX response.
- AJAX PHP.
- AJAX Database.

¿Qué es AJAX?

- AJAX (**A**synchronous **J**avaScript **a**nd **X**ML).

  Conjunto de técnicas de desarrollo web que permiten que las aplicaciones web funcionen de forma asíncrona, procesando cualquier solicitud al servidor en segundo plano.

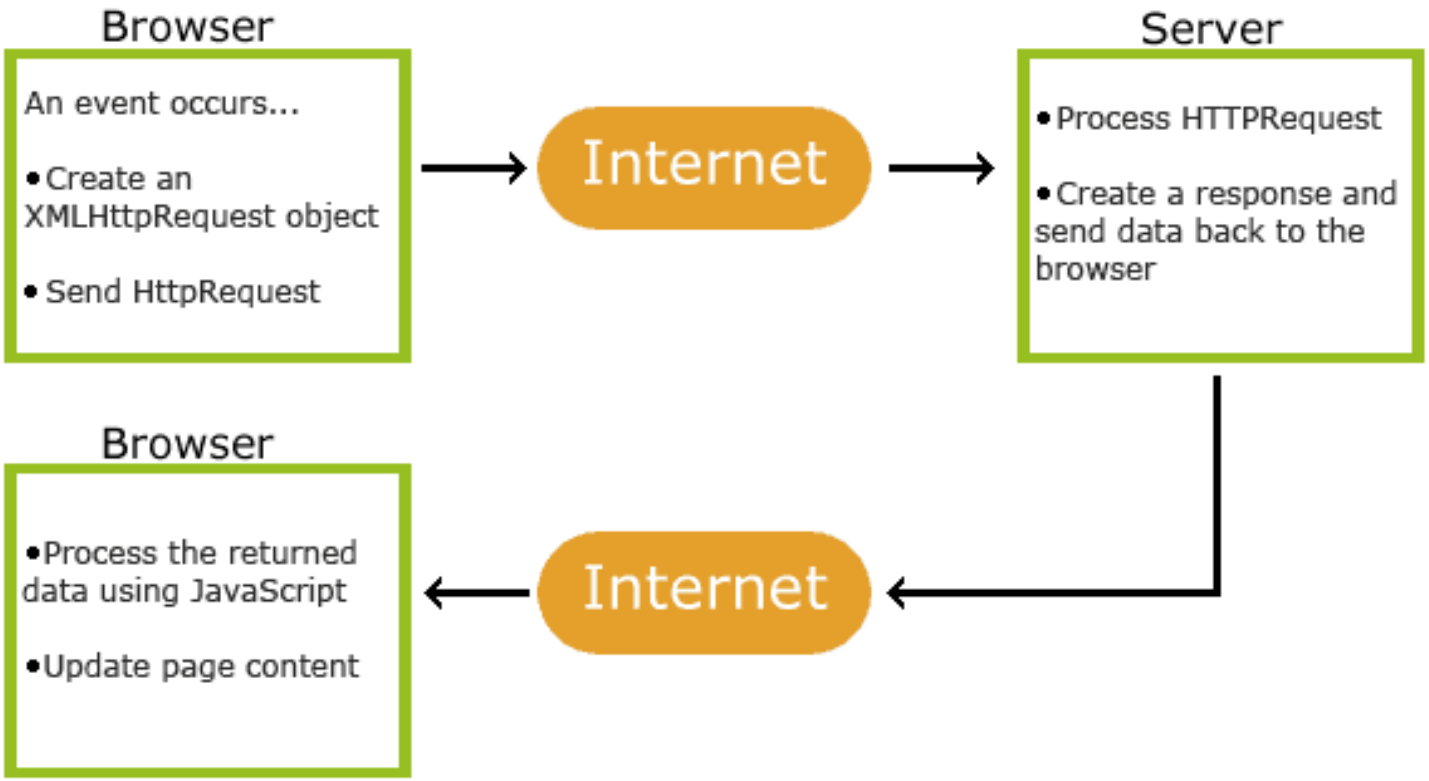
- Permite:
  - Leer datos de un servidor web, después de que se haya cargado la página
  - Actualizar una página web sin recargar la página
  - Enviar datos a un servidor web - en segundo plano

¿Qué es AJAX?

- **AJAX no es un lenguaje de programación.**
- AJAX usa una combinación de:
  - Un objeto XMLHttpRequest (nativo de js) integrado en el navegador (para solicitar datos de un servidor web). En nuestro caso usaremos la API Fetch (puede considerarse también nativa).
  - JavaScript y HTML DOM (para mostrar o utilizar los datos)
- También se pueden enviar datos en formato texto plano o JSON.

¿Qué es AJAX?

How AJAX Works



Aclaración. ¿Qué es API Rest?

- Es una interfaz que nos permite comunicarnos con otra aplicación que espera que hagamos una petición, puede ser para obtener datos, añadir datos, borrar…

- Las operaciones más importantes relacionadas con los datos en cualquier sistema REST y la especificación HTTP son cuatro: **POST** (crear), **GET** (leer y consultar), **PUT** (editar) y **DELETE** (eliminar).

4 formas de llamar a API REST

- **1. XMLHttpRequest (AJAX). Nativo.**
- **2. Fetch. Nativo.**
- **Axios. Librería Externa.**
- **jQuery.Ajax(). Librería Externa.**
- **Express.**

AJAX XMLHttp.

- La piedra angular de AJAX es el objeto XMLHttpRequest.
- Procedimiento general de comunicación con el servidor.
  - Crear un objeto XMLHttpRequest.
  - Definir una función de CallBack.
  - Abra el objeto XMLHttpRequest.
  - Enviar una solicitud a un servidor.

AJAX XMLHttp.

- **Crear un objeto XMLHttpRequest**.

```
variable = new XMLHttpRequest();
```

- **Definir una función de CallBack**. Una función Callback es una función que se pasa como parámetro a otra función.

```
xhttp.onload = function() {
  // What to do when the response is ready
}
```

- **Abra el objeto XMLHttpRequest y Enviar una solicitud a un servidor**.

```
xhttp.open("GET", "ajax_info.txt");
xhttp.send();
```

AJAX XMLHttp.

- Ejemplo:

```
<!DOCTYPE html>
<html>
<body>

<h2>The XMLHttpRequest Object</h2>

<div id="demo">
<p>Let AJAX change this text.</p>
<button type="button" onclick="loadDoc()">Change Content</button>
</div>

<script>
function loadDoc() {
  const xhttp = new XMLHttpRequest();
  xhttp.onload = function() {
    document.getElementById("demo").innerHTML = this.responseText;
  }
  xhttp.open("GET",“ejemploAJAX.txt");
  xhttp.send();
}
</script>

</body>
</html>
```

AJAX XMLHttp.

- Métodos del objeto XMLHttpRequest.

Method	Description
new XMLHttpRequest()	Creates a new XMLHttpRequest object
abort()	Cancels the current request
getAllResponseHeaders()	Returns header information
getResponseHeader()	Returns specific header information
open(*method, url, async, user, psw*)	Specifies the request *method*: the request type GET or POST *url*: the file location *async*: true (asynchronous) or false (synchronous) *user*: optional user name *psw*: optional password
send()	Sends the request to the server Used for GET requests
send(*string*)	Sends the request to the server. Used for POST requests
setRequestHeader()	Adds a label/value pair to the header to be sent

AJAX XMLHttp.

- Propiedades del objeto XMLHttpRequest.

Property	Description
onload	Defines a function to be called when the request is received (loaded)
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the status-number of a request 200: "OK" 403: "Forbidden" 404: "Not Found" For a complete list go to the Http Messages Reference
statusText	Returns the status-text (e.g. "OK" or "Not Found")

AJAX XMLHttp.

- Ejemplo onreadystatechange.

```
function loadDoc() {
  const xhttp = new XMLHttpRequest();
  xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
      document.getElementById("demo").innerHTML =
      this.responseText;
    }
  };
  xhttp.open("GET", "ajax_info.txt");
  xhttp.send();
}
```

AJAX request.

- Para enviar una solicitud al servidor hay que usar los metodos open y send del objeto XMLHttpRequest.

Method	Description
open(*method, url, async*)	Specifies the type of request *method*: the type of request: GET or POST *url*: the server (file) location *async*: true (asynchronous) or false (synchronous)
send()	Sends the request to the server (used for GET)
send(*string*)	Sends the request to the server (used for POST)

AJAX request.

- Ejemplos GET

```
xhttp.open("GET", "demo_get.asp");
xhttp.send();
```

```
xhttp.open("GET", "demo_get.asp?t=" + Math.random());
xhttp.send();
```

```
xhttp.open("GET", "demo_get2.asp?fname=Henry&lname=Ford");
xhttp.send();
```

AJAX request.

- Ejemplos POST

```
xhttp.open("POST", "demo_post.asp");
xhttp.send();
```

```
xhttp.open("POST", "ajax_test.asp");
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xhttp.send("fname=Henry&lname=Ford");
```

AJAX response.

- Propiedades de respuesta de servidor.

Property	Description
responseText	get the response data as a string
responseXML	get the response data as XML data

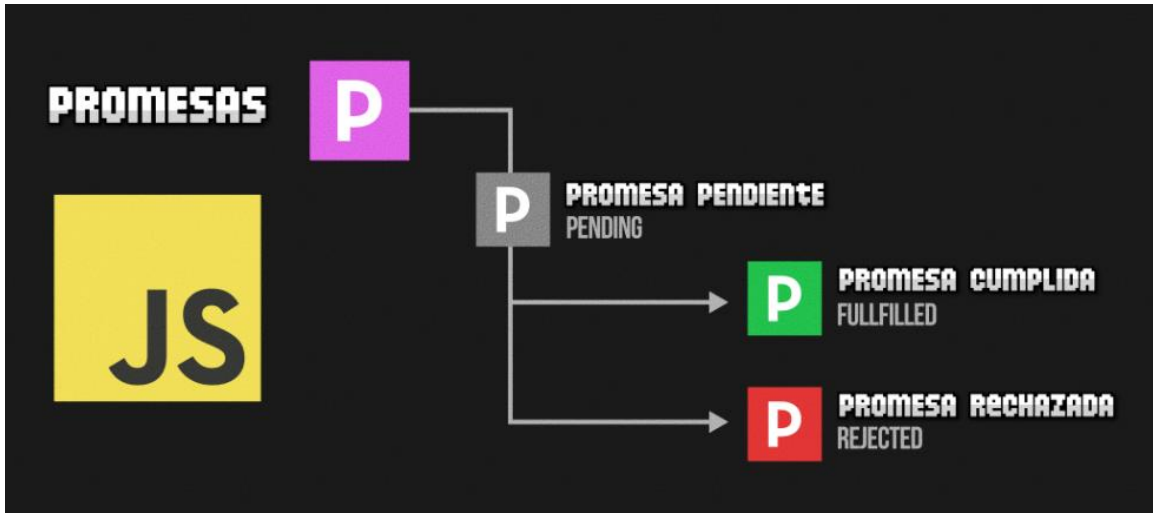
- El objeto XMLHttpRequest tiene incorporado un XML parser.

Fetch nativo

Promesas (Promise)

- Las **promesas** son un concepto para simplificar el manejo de la asincronía en Javascript.

- Como su propio nombre indica, en la programación al igual que en la vida real, una **promesa** es algo que en principio pensamos que se cumplirá en el futuro, pero a medida que pasa el tiempo pueden ocurrir varias cosas:



Promesas (Promise)

- Trabajar con promesas (thenneables)

Métodos	Descripción	Más info
`.then( FUNCTION resolve)`	Ejecuta la función cuando la promesa se cumple.	(Este artículo)
`.catch( FUNCTION reject)`	Ejecuta la función cuando la promesa se rechaza.	(Este artículo)
`.then( FUNCTION resolve, FUNCTION reject)`	Equivale a las 2 anteriores en el mismo `.then()`.	Ver `then() + catch()`
`.finally( FUNCTION end)`	Ejecuta la función cuando sale de pendiente.	Ver `finally()`

```
fetch("/robots.txt")
  .then(function(response) {
    // response: petición de respuesta del recurso solicitado
  })
```

Promesas

- Ejemplo básico Fetch.
  - En el caso del fetch(), si pedimos un archivo .json necesitaremos encadenar dos promesas, una para obtener la respuesta del servidor y otra para procesar el contenido JSON

```
fetch("/data.json")
  .then(response ⇒ {          // Gestiona la primera promesa
    return response.json();   // Devuelve otra promesa
  })
  .then(data ⇒ {              // Gestiona la segunda promesa
    console.log(data);        // Devuelve el JSON como un objeto ya procesado
  })
```

```
fetch("/data.json")
  .then(response ⇒ response.json())
  .then(data ⇒ console.log(data))
```

Fetch nativo

VER EJEMPLOS PRÁCTICOS